TP5. Echantillonnage et filtrage linéaire

I Objectifs

Explorer les notions d'échantillonnage et filtrage des signaux discrets.

1) Echantillonnage de signal et théorème de Shannon.

2) Conception de filtres linéaires.

II Preparation

- Reviser les sections 17, 18 et 19 du cours (Echantillonnage, systèmes LIT et filtrage linéaire)
- Le fichier `TP5data.zip` contient les fichiers utilisés dans le TP. Le télécharger depuis la page *chamilo* du cours.

III Echantillonnage

III-a Etude préliminaire

Nous nous intéressons dans cette partie à l'échantillonnage d'un signal sinusoïdal

$$x_i[k] = A_i \cos(2\pi f_{x_i} T_i k) \tag{1}$$

avec $k \in \{0,1,\ldots,K-1\}$, A_i l'amplitude et f_{x_i} est sa fréquence.

1) Charger le signal `x1` et `T1`. Le visualiser en fonction du temps.

2) Calculer et afficher le module de la transformée de Fourier de x_1. Quelle est la valeur de la fréquence f_{x_1} ?

3) Quelle est la fréquence d'échantillonnage minimum qu'on peut utiliser pour échantillonner le signal $x1$ sans repliement de spectre ?

4) Calculer $x_{s_1}[k] = x_1[k \times K_s]$ avec $K_s = 10$ et afficher le module de sa transformée de Fourier. Quelle est la fréquence de x_{s_1} ? **Astuce** calculer x_{s_1} à l'aide de `xs1=x1[::Ks]`.

5) Répéter les questions précédentes pour le signal `x2`. Que remarquez-vous ?

6) Superposer les signaux x_{s_1} et x_{s_2} et Expliquer les résultats précédents.

III-b Echantillonnage des signaux audio

Nous étudions dans cette partie l'effet d'échantillonnage sur les signaux audio. Pour rappel, nous chargeons les fichiers audio à l'aide de la fonction `scipy.io.wavfile.read()`

```
fs, x= scipy.io.wavfile.read('audiofile.wav')
print(f"sample rate= {fs}")
print(f"size={x.shape}")
```

Généralement les fichiers wav contiennent deux canaux (droit/gauche). Nous travaillons systématiquement avec le canal droit dans cette partie `xr=x[:,0]`.

1) Charger le fichier `Piccolo` et l'afficher en fonction du temps. Calculer et tracer le module de sa transformée de Fourier. Quelle la valeur du pitch ? Quelle est la fréquence la plus élevée dans le signal ?

2) Sous-échantillonner le signal avec un pas $K_s = 20$ et comparer le module de la transformée de Fourier du nouveau signal avec celui du signal original. Que remarquez-vous ?

3) Sauvegarder le fichier au format wav à l'aide de la fonction `scipy.io.wavfile.write("PiccoloEch.wav",fs//Ks,xs)`.

4) Ecouter le nouveau fichier et décrire l'effet de l'échantillonnage sur la qualité de signal.

5) Répéter les questions précédentes pour le fichier `SopranoPiano.wav`. Que remarquez-vous ?

IV Filtrage numérique

Nous nous intéressons dans cette partie à la conception de filtres numériques. La bibliothèque scipy fourni plusieurs outils pour concevoir et analyser la performance de filtres linéaire.

IV-a Filtre RII

Nous étudions dans cette partie les filtres à réponse impulsionnelle infinie (RII) à l'aide de la méthode de conception basée sur les familles paramétriques (Butterworth, Chebyshev, ...). Nous souhaitons concevoir un filtre passe-bas avec le cahier de charge suivant

$$A_p = -3dB, \text{(gain de la bande passante)}, \qquad A_c = -40dB, \text{(gain de la bande atténuée)}$$
$$f_p = 750Hz, \text{(fréquence de la bande passante)}, \qquad f_c = 1200Hz \text{(fréquence de la bande atténuée)}$$

Utiliser la valeur f_s du signal Piccolo.

1) Calculer l'ordre du filtre Butterworth pour satisfaire le cahier de charge précédent à l'aide de la fonction `scipy.signal.buttord`.

2) Calculer les coefficients du filtre à l'aide de la fonction `butter`. Tracer son diagramme de Bode à l'aide de la fonction `scipy.signal.dbode`.Est-ce que le cahier de charge est respecté ?

3) Calculer l'ordre du filtre Chebyshev II pour satisfaire le cahier de charge à l'aide de la fonction `scipy.signal.cheb2ord`. Que remarquez-vous ?

4) Calculer les coefficients du filtre à l'aide `scipy.signal.cheby2` et comparer son diagramme de Bode avec celui de Butterworth. Expliquer les différences.

Nous souhaitons utiliser le filtre précédent comme un filtre anti-repliement pour les signaux audios étudier dans la question précédente.

5) Appliquer les filtres Butterworth au signal Piccolo à l'aide de la fonction `scipy.signal.lfilter`

6) Comparer le module de la transformée de signal filtré et le signal original. Que remarquez-vous ?

7) Echantillonner le signal filtré et le sauvegarder au format wav. **Attention**, il faut convertir le signal filtré en `np.int16` avant de le sauvegarder au format Wav (`x.astype(np.int16)`) L'écouter et le comparer avec le signal échantillonné filtrage. Expliquer les différences.

IV-b Filtre RIF

Nous nous intéressons maintenant au à la conception des filtres à réponse impulsionnelle finie (RIF). Nous utilisons la méthode des fenêtres pour trouver les coefficients.

IV-b.1 Comparaison entre RIF et RII

1) Calculer les coefficients du filtre RIF à l'aide de la fonction `scipy.signal.firwin`. Utiliser le même ordre du filtre Butterworth conçu dans la partie précédente, f_s du signal Piccolo, f_p pour la fréquence de coupure.

2) Quels sont les coefficients de dénominateur ? Pourquoi ?

3) Comparer le digramme de Bode du filtre FIR avec celui de Butterworth. Expliquer les différences.

IV-b.2 Application au filtrage des signaux ECG

1) Charger le signal `ECG` et la fréquence d'échantillonnage `Fsecg` et l'afficher. Remarquer la composante basse fréquence qui contamine le signal ECG.

2) Concevoir un filtre passe-haut à l'aide de la méthode des fenêtres. Le filtre doit laisser passer un rythme cardiaque minimal de 40 battements par minute. Quelle est la fréquence de coupure correspondante ? Utiliser 15 coefficients dans le filtre et une fenêtre *boxcar*.

3) Afficher le diagramme de Bode. Est-ce que la performance du filtre vous semble acceptable ?

4) Appliquer le filtre au signal ECG. Est-ce que la composante basse fréquence est filtrée ?

5) Répéter les questions précédentes pour 145 coefficients dans le filtre FIR. Que remarquez-vous ?

6) Concevoir un filtre RII d'ordre 5 et de la famille elliptique. Comparer son diagramme de Bode avec celui de filtre RIF. Que remarquez-vous ?

7) Filtrer le signal ECG avec le filtre RII et comparer le signal filtré avec l'original.